



Learning Python - Sheet #7

Lists

Lists are very useful whenever we need to have multiple values, potentially an unknown number of them.

1. Lists

A list allows to store multiple values in a single variable. We can store any number of things in a list, including no things at all (empty list). These things can all be of the same type (e.g. all numbers) or can be of different types (e.g. mix of numbers, strings, lists, ...).

Note: Array is another type of data, very similar to lists. The main difference is that they only store the same kind of values (e.g. only numbers). Array are not very common in Python and sometimes people use the word “array” to talk about a “list”.

 **Pay attention:** In the code below, note the lines starting with #. They are **comments**. You don't have to type them, they will be ignored by Python. They are a way to document the code. Here, I use them to explain **what** the code does, but that's not how comments would be used in real code. A good use of comments is to explain **why** the code is written this way.

```
# Create an empty list:
numbers = []
while True:
    value = input("Enter a number: ")
    if value == "quit":
        break
    # If you don't enter an integer number nor 'quit', this will be very unhappy!
    numbers.append(int(value))

print("The sum of these numbers: %s is %s" % (numbers, sum(numbers)))

# There's no 'product' built-in. We could use math.product, but this shows looping on elements.
product = 1
for element in numbers:
    product *= element
print("Their product is: %s" % product)

# We can't get these if the list is empty (you can try removing the if to see what happens)
if len(numbers) > 0:
    print("Smallest number: %s - Biggest number: %s" % (min(numbers), max(numbers)))
    print("First element: %s - Last element: %s" % (numbers[0], numbers[-1]))

# We can't get these if the list has less than 3 elements (here again, you can try)
if len(numbers) >= 3:
    print("First three elements: %s" % numbers[:3])
    print("Last three elements: %s" % numbers[-3:])

# This is to illustrate len as well as accessing and modifying elements by their index.
for i in range(len(numbers)):
    numbers[i] += 1

print("After adding one to each number: %s" % numbers)
numbers.sort()
print("After sorting the list: %s" % numbers)
```

As said before, you can mix different element types in a list:

```
my_list = [3, 1.54, "python", [], ["this", "is", "a", "list"]]

for (i, element) in enumerate(my_list):
    print("Element #%s: %s" % (i, element))
```

 **Pay attention:** The function **enumerate** allows to get both the element and its index in the list. We could write **for element in my_list:** but then we wouldn't know the element's position in the list.

